

Architecture Produit Vivante

Web = Mobile · Modules Plugins · Vivant par Design

APV

Leopardo RH

Le manifeste qui dicte comment l'application vit, grandit et s'adapte

Conversationnelle
Home IA + Leo

Bi-surface
Web = Mobile

Modulaire
Plugins lazy

Vivante
Apprend & évolue

7 Chapitres · 48 tâches · Décisions architecturales complètes · Code de référence

SOMMAIRE

01	Le Manifeste	Pourquoi cette architecture · Les 4 piliers fondateurs · Ce que ça change
02	Principe Bi-surface	Web = Mobile · Parité totale · Grille partagée · Ce qui diffère légitimement
03	Architecture Modulaire Plugin	Lazy loading · Package Flutter · Route group Laravel · Contrat module
04	L'Application Vivante	Leo comme guide · Progressive disclosure · Auto-adaptation · Mise à jour sans friction
05	Home Screen Unifiée	Spécification complète web + mobile · Le pivot conversationnel
06	Toutes les tâches	T-APV-01 → T-APV-48 · 4 sprints · Priorités et dépendances
07	Les Lois	12 lois d'architecture · Immutables · Valent pour toujours

CH.01

LE MANIFESTE

Pourquoi cette architecture · Ce qu'elle change · Pour toujours

01

Un manifeste n'est pas une liste de règles techniques. C'est une déclaration de ce en quoi on croit — une boussole qu'on consulte quand on doute. L'Architecture Produit Vivante de Leopardo RH tient en une phrase :

"Une application est vivante quand elle grandit sans grossir, apprend sans être programmée à apprendre, et guide sans imposer."

Ce document est la source de vérité architecturale. Pas PILOTAGE.md. Pas un ticket Jira. Ce document. Toute décision qui contredit une loi posée ici doit d'abord modifier ce document — jamais l'inverse.

Les 4 piliers

Pilier	Principe	Conséquence concrète	Antipattern évité
01 — Conversationnelle	Leo IA est le point d'entrée principal. L'app parle avec l'utilisateur avant de lui imposer une interface.	Home = chat + alertes + chips. Le dashboard classique est une destination, pas un point de départ.	Dashboard KPIs figé que personne ne lit après J+7.
02 — Bi-surface	Le web et le mobile sont des citoyens de première classe. Pas de web 'simplifié' ni de mobile 'limité'.	Chaque feature est spécifiée et implémentée sur les deux surfaces avec la même richesse fonctionnelle.	Web = version bureau dégradée. Mobile = version légère pour les employés.
03 — Modulaire Plugin	Chaque module (Finance, Caméras, RH...) est un plugin autonome. L'app core ne connaît pas ses plugins.	Lazy loading Flutter. Route group isolé Laravel. Un utilisateur sans Finance ne télécharge pas le code Finance.	Monolithe UI où tout est importé même si non utilisé.
04 — Vivante	L'app enseigne, s'adapte au niveau de l'utilisateur et peut évoluer sans casser ce qui existe.	Progressive disclosure. Leo guide in-context. Modules ajoutables sans refonte. Mises à jour sans migration forcée.	App figée dans ses conventions initiales. Refonte nécessaire pour chaque évolution majeure.

Ce que cette architecture change concrètement

Pour chaque future décision de développement, ces 5 questions doivent être posées dans cet ordre :

- 1. Est-ce disponible sur les deux surfaces ?** Si une feature est définie seulement pour mobile ou seulement pour web, c'est un bug de spec, pas une décision.
- 2. Est-ce dans le bon module ?** Chaque ligne de code appartient à un module identifié. Le core ne contient que la navigation, l'auth et Leo. Tout le reste est un plugin.
- 3. Est-ce que ça enseigne ?** Une nouvelle feature doit être découvrable sans formation. Si Leo ne peut pas l'expliquer en 1 phrase, elle est trop complexe.
- 4. Est-ce que ça peut évoluer sans casser ?** Les contrats API sont versionnés. Les composants UI sont paramétrables. Les données sont extensibles via JSONB.
- 5. Est-ce que ça respecte les 12 Lois ?** Voir Chapitre 07. Si non → refuser la PR, pas modifier les Lois.

CH.02

PRINCIPE BI-SURFACE

Web et Mobile traités à égalité absolue · Parité totale

02

Web ≠ version bureau de l'app mobile. Mobile ≠ version allégée du web. Ce sont deux surfaces différentes du même produit, avec les mêmes fonctionnalités, adaptées aux contraintes de chaque contexte (curseur vs pousse, largeur vs hauteur, permanence vs mobilité).

Ce qui est identique entre les deux surfaces

- **Toutes les fonctionnalités** : si on peut créer une facture sur mobile, on peut en créer une sur web. Si on peut voir une caméra sur web, on peut la voir sur mobile.
- **Le langage visuel** : mêmes couleurs de domaine, mêmes typographies, mêmes niveaux de hiérarchie. Un utilisateur qui passe du mobile au web reconnaît immédiatement l'environnement.
- **Leo IA** : la Home conversationnelle existe sur les deux surfaces. Pas de Home IA sur mobile et dashboard classique sur web.
- **Les modules et leur activation** : le plan Business active Finance sur mobile ET sur web. Simultanément. Il n'y a pas de 'version web' et 'version mobile' du plan.
- **Les données en temps réel** : une facture créée sur mobile est immédiatement visible sur web. Un pointage depuis mobile apparaît sur web sans délai.

Ce qui diffère légitimement

■ MOBILE (Flutter)

Format portrait (375-430px wide)
Touch targets 44dp minimum
BottomNavigationBar (max 5 tabs)
 Swipe gestures → actions
 Notifications push natives
 Caméra / photo intégrée
 Biométrie FaceID/TouchID
 Offline partiel (cache local)
 Navigation GoRouter (Flutter)
 Composants Material 3 / Cupertino

■ WEB (Blade + Alpine.js)

Format paysage (1024px+ wide)
Click targets 32px minimum
Sidebar + topnav (layout 2 colonnes)
 Hover states sur tous les éléments
 Keyboard shortcuts (■K pour Leo)
 Drag & drop (listes, colonnes)
 Multi-fenêtres simultanées
 Tables denses avec plus de colonnes
 Navigation Laravel routes (Blade)
 Tailwind CSS + Alpine.js

Les différences légitimes sont toutes liées au contexte d'utilisation — jamais à la richesse fonctionnelle.

Grille de référence partagée

Élément design	Mobile Flutter	Web Blade	Source de vérité
Couleur RH	AppColors.rh = #10B981	text-emerald-500 / bg-emerald-50	AppColors.dart → tailwind.config.js
Couleur Finance	AppColors.finance = #F59E0B	text-amber-500 / bg-amber-50	AppColors.dart → tailwind.config.js
Couleur Sécurité	AppColors.security = #3B82F6	text-blue-500 / bg-blue-50	AppColors.dart → tailwind.config.js
Couleur IA/Leo	AppColors.ia = #7C3AED	text-violet-600 / bg-violet-50	AppColors.dart → tailwind.config.js
Police principale	Inter (GoogleFonts)	Inter (Google Fonts CDN)	TYPOGRAPHY.md
Rayon de bordure	BorderRadius.circular(8)	rounded-lg (8px)	—
Badge statut présent	LeopardoBadge.present		STATUTS.md
Badge statut absent	LeopardoBadge.absent		STATUTS.md
Espace vertical S	8dp	space-y-2 (8px)	—
Espace vertical M	16dp	space-y-4 (16px)	—
Espace vertical L	24dp	space-y-6 (24px)	—

RÈGLE : Quand une couleur ou un token change, il change dans AppColors.dart ET dans tailwind.config.js dans la même PR. Le fichier STATUTS.md liste les valeurs officielles de chaque statut sur les deux surfaces. C'est lui qui fait foi.

Composants partagés vs natifs — décision matrix

Composant	Mobile	Web	Partagé comment
Badge statut	LeopardoBadge	x-attendance-badge	Mêmes couleurs, même sémantique, implémentation différente
Barre de chat Leo	ChatInputBar	x-chat-input (Alpine.js)	Même UX (expand, envoi, placeholder), code différent
Quick Actions chips	QuickActionsRow	x-module-chips	Même ordre, mêmes couleurs domaines, scroll si overflow
Alert banner	AlertBanner	x-alert-banner	Mêmes 3 types (urgent/warning/info), mêmes couleurs
KPI card finance	FinanceMetricCard	x-finance-kpi	Même layout, même truncation montant, couleur domaine
Empty state	EmptyState	x-empty-state	Même message, même illustration SVG, même CTA
Modal / Dialog	showModalBottomSheet / AlertDialog	Alpine.js modal	Design cohérent, implémentation plateforme
Skeleton loading	ShimmerWidget	CSS skeleton animation	Même structure que le contenu réel

CH.03

ARCHITECTURE MODULAIRE PLUGIN

Lazy loading · Packages indépendants · Contrat module

03

Définition : un module est un ensemble cohérent de features (Finance, Caméras, RH...) qui peut être activé, désactivé, et mis à jour indépendamment du reste de l'application. Un utilisateur dont le plan n'inclut pas le module Finance ne télécharge pas, n'initialise pas, et n'exécute pas une seule ligne du code Finance.

Vue d'ensemble — l'app comme écosystème de plugins



Le core Leopardro (centre) gère l'auth, Leo IA, la navigation, et le registre des modules. Chaque plugin se connecte au core via un contrat standard.

Structure Flutter — package modulaire

Structure packages Flutter

```
leopardo_rh/
├── packages/
│   ├── leopardo_core/           # Auth, navigation, Leo, design tokens
│   │   ├── lib/core/theme/      # AppColors, AppTheme, Typography
│   │   ├── lib/core/navigation/ # AppShell, ModuleRegistry, GoRouter core
│   │   ├── lib/core/leo/        # ChatInputBar, ChatMessage, HomeBloc
│   │   ├── lib/core/widgets/    # LeopardroBadge, EmptyState, AlertBanner
│   │   └── module_rh/           # Module RH — package indépendant
│   │       ├── lib/screens/      # AttendanceScreen, HistoryScreen, EmployeeList
│   │       ├── lib/blocs/        # AttendanceBloc, EmployeeBloc
│   │       └── lib/module.dart    # RhModule.register() — point d'entrée
│   ├── module_finance/         # Module Finance — package indépendant
│   │   ├── lib/screens/         # InvoiceList, ExpenseCreate, TreasuryScreen...
│   │   ├── lib/blocs/           # InvoiceBloc, FinanceDashboardBloc...
│   │   └── lib/module.dart       # FinanceModule.register()
│   ├── module_cameras/         # Module Caméras — package indépendant
│   │   ├── lib/screens/         # CameraListScreen, CameraViewScreen...
│   │   ├── lib/blocs/           # CameraBloc, StreamTokenBloc
│   │   └── lib/module.dart       # CameraModule.register()
│   └── apps/
│       ├── leopardo_app/        # App principale — compose les packages
│       └── pubspec.yaml          # Dépend de leopardo_core + modules actifs
```

Le contrat Module — interface standard

```
LeopardoModule — contrat standard
// packages/leopardo_core/lib/core/navigation/module_contract.dart
abstract class LeopardoModule {
  // Identifiant unique du module
  String get id; // 'finance', 'cameras', 'rh'
  String get label; // 'Finance', 'Caméras', 'RH & Pointage'
  IconData get icon; // Icons.account_balance_outlined
  Color get domainColor; // AppColors.finance
  String get minPlan; // 'starter', 'business', 'enterprise'

  // Routes que ce module contribue au GoRouter
  List<RouteBase> get routes;

  // Onglet BottomNav (null si pas d'onglet)
  AppTab? get bottomNavTab;

  // Alertes que ce module peut générer pour la Home
  Future<List<HomeAlert>> getAlerts(BuildContext context);

  // Questions Leo que ce module peut répondre
  List<String> get leoTopics; // ['factures', 'dépenses', 'trésorerie']

  // Quick action chip pour la Home
  ModuleChip get homeChip;
}

// Enregistrement dans main.dart — lazy : ne charge que si plan l'autorise
ModuleRegistry.registerAll([
  RhModule(), // Toujours chargé
  if (plan.hasModule('finance')) FinanceModule(),
  if (plan.hasModule('cameras')) CameraModule(),
  if (plan.hasModule('alarms')) AlarmModule(), // Phase 2
]);
```

Structure Laravel — route groups isolés

Structure Laravel modulaire

```

api/
  app/Http/Controllers/
    Core/           # Auth, Leo IA, Notifications, Settings
    Modules/
    Rh/             # EmployeeController, AttendanceController
    Finance/        # InvoiceController, ExpenseController...
    Cameras/        # CameraController, StreamTokenController
  routes/
    api.php         # Include les routes modules selon plan
    modules/
    rh.php          # Préfixe /api/v1/rh/ - middleware [sanctum,tenant]
    finance.php     # Préfixe /api/v1/finance/ - + [finance.subscription]
    cameras.php     # Préfixe /api/v1/cameras/ - + [camera.subscription]
  resources/views/
    layouts/        # app.blade.php, guest.blade.php
    home/           # home/index.blade.php (Home conversationnelle WEB)
    modules/
    rh/             # employees/, attendance/
    finance/        # invoices/, expenses/, treasury/
    cameras/        # index.blade.php, view.blade.php
    components/     # x-attendance-badge, x-chat-input...

```

Règles d'or de la modularité

- **Le core n'importe jamais un module.** Si leopardo_core a une dépendance sur module_finance, l'architecture est cassée.
- **Les modules ne se connaissent pas entre eux.** Finance ne peut pas importer Camera. Ils communiquent via des événements du core (EventBus ou Riverpod ref).
- **Chaque module est testable seul.** flutter test packages/module_finance/ doit fonctionner sans les autres modules.
- **Ajouter un module = 1 package + 1 ligne d'enregistrement.** Rien d'autre ne change dans l'app.
- **Supprimer un module = 1 ligne d'enregistrement.** L'app compile toujours. L'utilisateur voit que le chip a disparu.

'Vivante' ne signifie pas que l'app change toute seule. Ça signifie qu'elle s'adapte au niveau de l'utilisateur, qu'elle enseigne en fonctionnant, et qu'elle peut évoluer techniquement sans forcer les utilisateurs à réapprendre.

Le parcours d'apprentissage — 5 stades

Découverte

1

J0-J3 Leo guide les premières actions

Routine

2

J4-J14 Habitudes formées

Maîtrise

3

J15-J30 Shortcuts découverts

Expert

4

J31+ Personnalise son workflow

Ambassadeur

5

J60+ Recommande à d'autres

L'interface révèle ses fonctionnalités progressivement selon le stade de l'utilisateur.

Progressive Disclosure — ce qu'on montre et quand

Stade	Ce qui est visible	Ce qui est masqué	Déclencheur pour révéler
J0 — Découverte (0 action)	Home Leo + 3 chips max + 1 alerte si existante. Formulaires en mode simple (champs essentiels).	Filtres avancés. Exports. Config avancée. Rapport. Accès tiers caméras.	Automatique — tout le monde commence ici.
J1-J7 — Formation (10+ actions)	+ Filtres basiques. + Historique. + Badge progression (% complet du profil).	Exports. Config avancée. Intégrations. Analytics.	10 actions réalisées dans l'app (compteur HomeBloc).
J8-J30 — Maîtrise (50+ actions)	+ Exports CSV. + Raccourcis clavier web (■K Leo). + Bulk actions.	Config avancée API. Intégrations tiers.	50 actions ou 7 jours d'utilisation consécutifs.
J31+ — Expert (illimité)	Toutes les features du plan. Dashboard analytique. Personnalisation chips. API publique si Enterprise.	—	30 jours d'utilisation active.

Implémentation : un compteur d'actions est stocké dans le profil utilisateur (employees.metadata JSONB, clé 'app_actions_count'). L'API retourne le stade courant dans /api/v1/auth/me sous la clé 'learning_stage'. Les composants Flutter et Blade lisent ce stade pour décider ce qu'ils affichent.

Leo comme guide — 4 modes d'enseignement

Mode Leo	Déclenchement	Exemple	Durée
Contextuel passif	L'utilisateur ouvre un écran pour la première fois	Ouvre InvoiceCreateScreen → Leo envoie un message : 'Vous créez votre première facture. Les champs marqués * sont obligatoires. Commencez par le nom du client.'	1 fois par écran
Réponse à l'action	L'utilisateur réalise une action significative	Envoie une facture → Leo : 'Facture INV-2026-0001 envoyée à client@email.com. Elle apparaîtra en 'Envoyée' jusqu'au paiement. Vous pouvez la relancer dans 7 jours.'	À chaque action majeure
Proactif matinal	Première ouverture de l'app du jour	Leo affiche dans la Home : 'Bonjour Karim. 3 choses pour aujourd'hui : 1 facture en retard, 2 dépenses à valider, 1 employé absent non justifié.'	1 fois par jour à la première ouverture
Guide à la demande	L'utilisateur tape une question à Leo	'Comment modifier une facture déjà envoyée ?' → Leo explique la procédure exacte dans l'interface actuelle avec un lien deep link vers l'écran.	Illimité (quota selon plan)

Évolutivité sans friction — les 4 garanties

01

Backward compatibility

Chaque endpoint API a un numéro de version (/api/v1/). Quand un changement breaking est nécessaire, /api/v2/ est créé. V1 reste actif pendant 6 mois minimum. L'app mobile peut tourner en V1 pendant que le web est déjà en V2.

02

JSONB extensible

Toutes les tables ont un champ metadata JSONB (employees.metadata, companies.metadata, invoices.metadata...). Les nouvelles données d'un module peuvent être stockées dans ce JSONB sans migration de schéma. La migration formelle se fait en Phase suivante.

03

Feature flags

Chaque feature non-stable est derrière un feature flag (companies.features JSONB). Le flag peut être activé par plan, par entreprise, ou globalement depuis le super-admin. Un bug = désactiver le flag. Pas de hotfix en urgence.

04

Modules hot-swappable

Un module peut être mis à jour indépendamment. La mise à jour du module Finance ne nécessite pas de publier une nouvelle version de l'app entière sur les stores. Code push (Shorebird) pour les corrections non-natives.

Anatomie commune — les zones partagées

Zone	Mobile	Web	Partagé ?
Salutation personnalisée	AppBar : 'Bonjour [Prénom]' + avatar	Header : 'Bonjour [Prénom]' + avatar + date	Contenu identique
Alertes contextuelles	AlertBanner stack (max 3)	Bandeau horizontal (max 3, dismissable)	Même logique, même contenu
Interface Leo	ChatListView (flex) + ChatInputBar	Panneau latéral gauche (1/3 width) ou modal	Même API, même historique
Quick Actions	QuickActionsRow — scroll horizontal	Module nav sidebar + chips en haut du contenu	Même modules, même couleurs
Contenu dynamique	Pas de dashboard KPIs en Home	Pas de dashboard KPIs en Home	Identique : l'app ne force pas le dashboard
Raccourcis	Long press chip → action directe	■K pour ouvrir Leo, ■N pour nouvelle action	Fonctionnalité différente, philosophie identique

Layout web — Home conversationnelle en deux colonnes

```
home/index.blade.php – layout deux colonnes
<!-- resources/views/home/index.blade.php -->
<div class='min-h-screen bg-slate-50 flex'>
  <!-- Panneau Leo (1/3) - toujours visible sur desktop -->
  <aside class='w-80 bg-white border-r border-slate-200 flex flex-col'>
    <x-leo-header :user='auth()->user()' />
    <x-alert-banners :alerts='$alerts' />
    <div class='flex-1 overflow-y-auto p-4'>
      <x-chat-messages :messages='$messages' />
    </div>
    <x-chat-input placeholder='Posez votre question à Leo...' />
    <x-module-chips :modules='$activeModules' :max='5' />
  </aside>
  <!-- Zone contenu principale (2/3) -->
  <main class='flex-1 p-8'>
    <!-- S'adapte selon l'état : vide = welcome, actif = contenu module -->
    @if($activeModuleContent)
      @include('modules.'. $activeModule.'.summary')
    @else
      <x-home-welcome :user='auth()->user()' :stage='$learningStage' />
    @endif
  </main>
</div>
```

États de la Home — machine d'états complète

État	Condition	Mobile affiche	Web affiche
PREMIER LANCEMENT	app_actions_count = 0	Message de bienvenue Leo + 3 chips + 0 alertes	Welcome panel + Leo guide + chips
MATIN (urgences)	Première ouverture du jour + alertes actives	1-3 AlertBanners + recap Leo proactif	Bandeau alertes + résumé Leo en sidebar
ACTIF SANS ALERTE	Utilisation normale, 0 alerte	Historique chat + chips + input	Chat sidebar + contenu dernier module visité
EN ATTENTE	Après envoi message Leo	TypingIndicator (3 dots) + input désactivé	Même en sidebar
MODULE ACTIF	Tap sur un chip	GoRouter → écran du module	Contenu module en zone principale
HORS LIGNE	Pas de connexion	Banner 'Mode hors ligne' discret + cache messages	Banner + cache + actions indisponibles grisées
QUOTA ATTEINT	Messages Leo épuisés pour la journée	Message Leo: 'Quota journalier atteint. [Upgrade]'	Même message + lien vers plans

Personnalisation de la Home par rôle

Élément	Manager Principal	Gestionnaire Finance	RH Manager	Employé simple
Chips ordre (gauche→droite)	RH · Finance · Cam · Rapports · ...	Finance · Factures · Dépenses · ...	RH · Pointage · Absences · ...	Pointer · Mes Congés · Ma Fiche
Alertes montrées	Toutes	Finance uniquement	RH uniquement	Ses données uniquement
Leo scope	Tout Leopardo	Finance + App (0 RH)	RH + App (0 Finance)	Ses données personnelles
Zone contenu web	Dernier écran ou KPIs globaux	Dashboard Finance	Dashboard RH + présences	Résumé personnel (pointage du jour)
Raccourci web ■N	Choix : Employé / Facture / Dépense	Choix : Facture / Dépense / BC	Choix : Employé / Absence	Pointer l'arrivée directement

CH.06

TOUTES LES TÂCHES — T-APV-01 À T-APV-48

4 sprints · Priorités · Dépendances · Surfaces

06

Ces tâches sont ADDITIONNELLES aux tâches des modules (Finance, Caméras, etc.). Elles couvrent uniquement l'architecture core : la modularité, la bi-surface, Leo, et le système vivant. Durée estimée : 3-4 semaines à 2h/jour en parallèle du développement des modules.

Sprint 1 — Architecture core et fondations (Semaine 1)

T-APV-01

Package leopardo_core Flutter · 1 j · Critique

Créer le package séparé. Migrer depuis lib/core/ : AppColors, AppTheme, ModuleRegistry, AppShell, LeopardoBadge, ErrorMessage, EmptyState. pubspec.yaml local path reference dans l'app principale.

T-APV-02

LeopardoModule abstract class Flutter · 0.5 j · Critique

Créer le contrat module (section Ch.03). id, label, icon, domainColor, minPlan, routes, bottomNavBar, getAlerts(), leoTopics, homeChip. Documenter dans leopardo_core/README.md.

T-APV-03

ModuleRegistry v2 Flutter · 0.5 j · Critique

Refactorer pour utiliser LeopardoModule. registerAll(). tabsForPlan(). alertsForUser(). chipsForHome(maxVisible:4). leoTopicsForUser(). Supprimer tous les hardcodes.

T-APV-04

Package module_rh Flutter · 0.5 j · Critique

Créer le package. Migrer AttendanceScreen, HistoryScreen, EmployeeListScreen, tous les blocs RH. RhModule implements LeopardoModule. Connecter au ModuleRegistry.

T-APV-05

Route groups Laravel isolés Web Blade · 0.5 j · Critique

Créer routes/modules/rh.php, finance.php, cameras.php. Inclure dans api.php selon plan. Vérifier que les routes sont correctement préfixées et que les middlewares sont indépendants.

T-APV-06

config/modules.php Laravel Web Blade · 0.25 j · Critique

Configuration centralisée des modules web (équivalent ModuleRegistry Flutter). Chaque module : id, label, icon, min_plan, color, routes_prefix, sidebar_group. Source de vérité unique.

T-APV-07

metadata JSONB sur tables clés Web Blade · 0.5 j · Haute

Ajouter metadata JSONB DEFAULT '{}' sur : employees, companies, invoices, expenses. Migration Laravel. Ajout des clés initiales : employees.metadata.app_actions_count, employees.metadata.learning_stage.

T-APV-08

Feature flags system Web Blade · 0.5 j · Haute

Service FeatureFlag::enabled('feature_name', \$user). Stocké dans companies.features JSONB. Super-admin peut toggle par company. Retourné dans /api/v1/auth/me.features. Utilisé par les modules pour activer/désactiver des sous-features.

T-APV-09**LearningStageService Web Blade · 0.5 j · Haute**

Service qui calcule le stade (discovery/routine/mastery/expert) depuis `employees.metadata.app_actions_count`. Incrément du compteur à chaque action via middleware `ActionTrackerMiddleware` léger. Retourné dans `/api/v1/auth/me`.

T-APV-10**Tests architecture packages Flutter · 0.5 j · Haute**

Vérifier que `leopardo_core` n'importe aucun module. Vérifier que `module_finance` n'importe pas `module_rh`. Test flutter pub get dans chaque package isolément. CI job 'package-isolation' dans `tests.yml`.

Sprint 2 — Home conversationnelle bi-surface (Semaine 2)**T-APV-11****HomeConversationalScreen Flutter Flutter · 1.5 j · Critique**

Écran complet selon Ch.05. AppBar salutation. AlertBanners (AnimatedSize). ChatListView (Expanded). ChatInputBar. QuickActionsRow. SafeArea. Branché sur HomeBloc.

T-APV-12**HomeBloc v2 avec learning stage Flutter · 1 j · Critique**

States: HomeInitial, HomeLoading, HomeLoaded(alerts,messages,learningStage,visibleChips). Calcule les chips visibles selon learningStage et plan. Proactive morning message si J0.

T-APV-13**Home conversationnelle web (Blade) Web Blade · 1.5 j · Critique**

Layout 2 colonnes (Ch.05). x-leo-header, x-alert-banners, x-chat-messages, x-chat-input, x-module-chips. Zone contenu adaptative. Responsive : Leo devient modal sur tablette/mobile web.

T-APV-14**AiChatService Laravel v2 Web Blade · 1 j · Critique**

Refactorer pour injecter `learning_stage` et `active_modules` dans le contexte. Ajouter `getAlerts()` dans le contexte proactif. Support multi-provider (OpenAI / Anthropic) via config('ai.provider').

T-APV-15**Progressive disclosure Flutter Flutter · 0.75 j · Haute**

`LearningStageProvider` (Riverpod). `shouldShow(feature, stage) → bool`. Wraps les éléments avancés (filtres, exports, config). Lire depuis `auth/me.learning_stage`. Pas de flutter_restart requis lors du changement de stade.

T-APV-16**Progressive disclosure Blade Web Blade · 0.75 j · Haute**

@if(\$user->learningStage->canSee('advanced_filters')) ... @endif. Blade directive @whenStage(). CSS opacity-50 sur éléments pré-révélés (visible mais indiqué comme 'bientôt disponible').

T-APV-17**ActionTrackerMiddleware Web Blade · 0.5 j · Haute**

Middleware léger qui incrémente `employees.metadata.app_actions_count` après chaque requête POST/PATCH/DELETE réussie. Ne bloque jamais la requête même en cas d'erreur du tracker. Batch update toutes les 5 actions.

T-APV-18**Leo guide contextuel (premier usage) Flutter · 0.75 j · Haute**

FirstUseGuide service : track quels écrans ont été ouverts pour la première fois (`metadata.first_seen_screens`). Sur premier accès → `HomeBloc.add(SendLeoContextualGuide(screenId))`. Messages pré-écrits par écran.

T-APV-19 **Leo guide contextuel web** Web Blade · 0.5 j · Haute

Session-based : première visite d'un écran → injecter un message Leo en sidebar. Alpine.js store pour ne pas re-afficher si déjà vu dans la session. Cohérent avec Flutter.

T-APV-20 **Tests Home bi-surface** Transversal · 1 j · Haute

Flutter : widget test HomeConversationalScreen (états loading/loaded/offline/quota). Pest : test home/index route (alertes injectées, modules filtrés par plan). E2E : ouvrir app → voir Home → envoyer message → voir réponse Leo.

Sprint 3 — Modularité et évolutivité (Semaine 3)

T-APV-21

Package module_finance Flutter · 0.5 j · Critique

Migrer tout le code Finance dans le package isolé. FinanceModule implements LeopardoModule. Vérifier que l'app compile avec et sans ce package.

T-APV-22

Package module_cameras Flutter · 0.5 j · Critique

Migrer tout le code Caméras. CameraModule implements LeopardoModule. flutter_webrtc uniquement dans ce package.

T-APV-23

Lazy loading Flutter (deferred) Flutter · 1 j · Haute

Utiliser dart deferred imports pour les packages modules non-essentiels. Le code Finance n'est chargé que si plan.hasModule('finance') ET l'utilisateur navigue vers Finance. Mesurer APK size avant/après.

T-APV-24

Feature flags Flutter consumer Flutter · 0.5 j · Haute

FeatureFlagProvider (Riverpod). featureEnabled('invoice_recurring') → bool. Lu depuis auth/me.features. Utilisé dans les modules pour conditionner l'affichage de sub-features.

T-APV-25

JSONB metadata extensibilité test Web Blade · 0.5 j · Haute

Créer un test qui vérifie qu'une nouvelle clé peut être ajoutée à employees.metadata sans migration. Documenter le pattern dans CONTRIBUTING.md : 'nouvelles données non-critiques → JSONB, données critiques → colonne'.

T-APV-26

API versioning V1 → V2 ready Web Blade · 0.75 j · Haute

Créer routes/api_v2.php vide mais structuré. ApiVersionMiddleware qui route vers v1 ou v2 selon Accept-Version header. Documenter la procédure de migration d'un endpoint de V1 à V2 dans docs/.

T-APV-27

Module hot-swap demo (Shorebird ready) Flutter · 0.5 j · Moyenne

Préparer l'app pour Shorebird code push. shorebird.yaml config. Documenter la procédure de publication d'une mise à jour d'un module sans passer par les stores. Garder en tête la contrainte iOS (pas de code natif).

T-APV-28

Raccourcis clavier web (■K, ■N) Web Blade · 0.75 j · Haute

Alpine.js global keyboard listener. ■K → focus ChatInput Leo. ■N → modal 'Nouvelle action' (Facture / Employé / Dépense selon modules actifs). Escape → ferme modal. Documenter dans le guide utilisateur.

T-APV-29

Mode hors-ligne gracieux Flutter Flutter · 0.75 j · Haute

connectivity_plus. Si offline : banner discret. Cache Hive pour les derniers messages Leo et les listes (employés, factures). Quick Actions visibles mais modules marqués 'hors ligne'. Sync automatique au retour online.

T-APV-30

Tests d'isolation modules Transversal · 0.75 j · Haute

CI job qui vérifie : leopardo_core ne dépend d'aucun module. Chaque module compile seul (flutter pub get + flutter build apk --debug). Laravel : chaque routes/modules/*.php est syntaxiquement valide isolément.

Sprint 4 — Polish système vivant (Semaine 4)

T-APV-31	Morning brief Leo automatique Flutter · 0.75 j · Haute HomeBloc.loadHome() : si première ouverture du jour, appeler /api/v1/ai/morning-brief qui retourne un résumé des urgences formaté pour Leo. Afficher comme premier message Leo de la journée.
T-APV-32	Suggestions Leo contextuelles Flutter · 0.5 j · Haute Sous la barre chat : Row de 2-3 suggestions selon le rôle, l'heure, et les alertes actives. Générées côté serveur dans /api/v1/ai/suggestions. Tap → auto-envoi la suggestion.
T-APV-33	Empty states intelligents Transversal · 0.5 j · Haute Les empty states affichent une suggestion Leo : 'Je peux vous aider à créer votre première facture. Dites-moi 'créer une facture' ou appuyez sur le bouton.' Lien direct entre empty state et Leo.
T-APV-34	Onboarding checklist (J0) Transversal · 0.75 j · Haute Pour les utilisateurs en stade 'discovery' : checklist flottante avec 5 actions de démarrage (Ajouter un employé, Faire un premier pointage, Créer une facture...). Dismiss une fois complété. Progress bar visible.
T-APV-35	Notification système intelligent Flutter · 0.5 j · Haute NotificationService : envoie la notification au bon moment (pas de spam). Max 3 notifications/jour. Groupées par module. Deep link vers l'entité concernée. Respect des préférences utilisateur (silence nocturne).
T-APV-36	Personnalisation chips (drag) Flutter · 0.75 j · Moyenne Long press sur un chip → mode réorganisation. Drag & drop l'ordre des chips. Ordre sauvegardé dans employees.metadata.preferred_chips_order. Appliqué à la prochaine ouverture.
T-APV-37	Métriques d'usage anonymes Web Blade · 0.5 j · Moyenne Compter (sans PII) : écrans les plus visités, questions Leo les plus posées, modules les plus utilisés par plan. Dashboard super-admin pour analyser. Guide les décisions produit Phase 2.
T-APV-38	Documentation vivante in-app Transversal · 0.75 j · Haute Chaque écran a un '?' icon → ouvre Leo avec le prompt 'Explique-moi cet écran'. Leo donne une explication contextuelle. Plus de PDF de documentation. La doc vit dans Leo.
T-APV-39	STATUTS.md et COULEURS.md Transversal · 0.25 j · Haute Créer deux fichiers de référence dans /docs : STATUTS.md (tous les statuts avec couleurs Flutter + Tailwind) et COULEURS.md (tous les tokens couleur). CI job qui vérifie que AppColors.dart et tailwind.config.js sont synchronisés.
T-APV-40	Regression tests bi-surface Transversal · 1 j · Haute golden_toolkit Flutter pour HomeScreen, AttendanceScreen, InvoiceList en light+dark. Dusk Laravel pour home web, employees, finance. Screenshots de référence commitées en repo. CI compare à chaque PR.
T-APV-41	Performance audit complet Transversal · 0.5 j · Haute Flutter : flutter analyze → 0 warnings. dart metrics check. Web : Lighthouse score > 90 sur Performance + Accessibility. API : EXPLAIN ANALYZE sur les 10 requêtes les plus fréquentes. Documenter les baselines.

T-APV-42 Guide contribution module Transversal · 0.5 j · Haute

CONTRIBUTING_MODULE.md : guide étape par étape pour créer un nouveau module. Template package Flutter. Template route group Laravel. Checklist PR pour les modules. Permet à un développeur externe de créer un module correctement.

T-APV-43 Accessibilité WCAG AA complète Transversal · 0.75 j · Haute

Flutter : Semantics sur tous les éléments interactifs. Contrast checker automatique. Web : axe-core dans CI. Tous les scores DevTools Accessibility > 90. Priorité : ChatInputBar, AlertBanners, Quick Action chips.

T-APV-44 Internationalisation architecture Transversal · 0.5 j · Moyenne

Flutter : ARB files par module (lib/l10n/ dans chaque package). Web : lang/ par module. Leo répond dans la langue de l'interface (injecté dans le contexte du prompt). Structure prête pour ajout de l'arabe RTL sans refonte.

T-APV-45 Monitoring erreurs front-end Transversal · 0.5 j · Haute

Flutter : Sentry.io integration. Capture les erreurs non-gérées avec contexte (module actif, action, état BLoC). Web : Sentry JS. Super-admin peut voir les erreurs groupées par module et version. Alerting si taux d'erreur > 1%.

T-APV-46 Changelog in-app Flutter · 0.5 j · Moyenne

Modal 'Nouveautés' affiché une fois par version (versionné via employees.metadata.last_seen_version). Liste les nouvelles features du module de l'utilisateur uniquement. Lien 'Demander à Leo' pour chaque feature listée.

T-APV-47 Load testing bi-surface Transversal · 0.5 j · Haute

k6 : 100 utilisateurs simultanés sur /home endpoint (alertes + suggestions Leo). Objectif < 500ms p95. Simuler 20 messages Leo simultanés. Documenter les limites et les scaling paths.

T-APV-48 APV.md — document vivant Transversal · 0.25 j · Critique

Créer /docs/APV.md qui résume ce manifeste en 1 page. Les 4 piliers. Les 12 Lois. Le contrat module. La grille bi-surface. Ce document est consulté à chaque PR. Mis à jour si une Loi évolue.

Sprint	Thème	Tâches	Effort estimé	Résultat
Sprint 1	Architecture core	10	~5 j-dev	Packages isolés, contrat module, feature flags
Sprint 2	Home bi-surface + Leo	10	~9 j-dev	Home conversationnelle web + mobile
Sprint 3	Modularité + évolutivité	10	~6,5 j-dev	Lazy loading, API versioning, mode offline
Sprint 4	Système vivant complet	18	~9 j-dev	Progressive disclosure, monitoring, contribution guide
TOTAL	Architecture Produit Vivante	48 tâches	~29,5 j-dev	Application vivante, bi-surface, modulaire

CH.07

LES 12 LOIS

Immutables · Valent pour toujours · Source de vérité absolue

Ces 12 Lois ne sont pas des recommandations. Ce sont des contraintes architecturales. Une décision qui viole une Loi doit d'abord modifier la Loi — avec justification écrite — avant d'être implémentée. Une PR qui viole une Loi sans modification de ce document est refusée automatiquement.

LOI 01

Home = Conversation

Le premier écran est le dashboard de K. Cette loi s'applique à tout module.

LOI 02

Web = Mobile en fonctionnalités

Toute feature doit être disponible sur Web et Mobile. Concernant uniquement les fonctionnalités.

LOI 03

Core n'importe aucun module

leopardo_core ne dépend d'aucun module. Caméras, RH, etc. ne dépendent que de Core.

LOI 04

Modules ne se connaissent pas

module_finances ne connaît pas module_logistique via des événements. Les interactions sont gérées via des interfaces inter-modules.

LOI 05

Couleur = Domaine

Vert = RH. Ambre = Finance. Les associations sont strictes. Une violation est interdite.

LOI 06

Leo enseigne, l'interface confirme

Leo est le canal principal de communication. Les modals confirment les actions. Elle est trop concise.

LOI 07

Grille partagée est la source de vérité

AppColors.dart est la source de vérité. CCULEURS.mn ne doit pas modifier l'autre.

LOI 08

Un module = un package

Chaque module doit être dans un package. Un module ne dépend jamais par impulsion d'un module.

LOI 09	Contrat module est immutable par release	L'interface Leop = major version pour migrer av
LOI 10	Données nouvelles → JSONB d'abord	Les nouvelles c JSONB metada est stabilisée (F
LOI 11	Progressive disclosure avant feature toggle	Avant de cache cachée via prog le système de s
LOI 12	Leo est spécialisé, pas généraliste	Leo répond uni de l'app). Le sy hors-domaine. dans ce manife

Ce manifeste est un document vivant. Il évolue avec le produit. Toute modification d'une Loi nécessite : (1) une justification écrite, (2) l'accord de l'équipe produit, (3) la mise à jour de /docs/APV.md. La date de modification est trackée dans le git blame de ce fichier. Version 1.0 — Avril 2026.